

Z

Programming Language

Reference Manual

Version 50.00

Successor to v48.92

ZinterPL Project · 2026

This manual describes the Z programming language as implemented by the Zim interpreter. It constitutes the authoritative specification of the language syntax, data model, control flow, and standard input/output primitives. No prior familiarity with the interpreter source is assumed.

Preface

Z is a statically typed, interpreted programming language designed for clarity, compactness, and portability across the widest possible range of computing environments. Its interpreter — Zim — is a single, self-contained C translation unit. A conforming implementation requires only a C89-capable compiler and the standard C library; no external dependency, build system, or runtime support is needed beyond that.

The design philosophy of Z stands in deliberate opposition to the prevailing trend of ever-expanding language surface area. Every construct in Z exists because it carries unambiguous meaning and measurable utility; no construct exists for mere convenience of the implementer. The result is a language that can be learned in its entirety in the course of a single afternoon and yet is expressive enough to implement non-trivial algorithms involving structured data, recursive function chains, and dynamic control flow.

Z source files carry the **.Zim** extension, which is also the informal name by which the language and its interpreter are jointly known. Library modules carry the **.Zlib** extension and are attached to a program at invocation time rather than embedded in the source, keeping the source uncontaminated and the library reusable across projects.

The memory model of Zim deserves particular mention. The entire virtual machine — variable tables, array and matrix storage registries, the program buffer, the state stack, the register file, and the scope chain — lives inside a single C structure of type **VM**. This structure can be statically allocated, requiring no heap at startup. Dynamic allocation occurs only when arrays and matrices are declared at runtime, and even then is strictly bounded by the declared sizes. The interpreter has been measured at a base footprint of approximately 284 kilobytes on Linux x86-64; it is explicitly targeted at the ESP32 microcontroller, where the entire VM fits comfortably within the available SRAM. No other interpreted language in common use makes the same claim.

This manual is organised as a reference rather than a tutorial. Each chapter addresses a single language construct and describes it completely, including its syntax, its semantics, and the invariants the programmer may rely upon. Where behaviour might otherwise be ambiguous, the specification is explicit even at the cost of some repetition.

Contents

- 1** Language Overview and Architecture
- 2** Lexical Conventions
- 3** Data Types and Variable Declaration
- 4** Arrays
- 5** Matrices
- 6** Assignment and Arithmetic Expressions
- 7** Increment and Decrement Operators
- 8** Functions: Declaration and Invocation
- 9** Return Expressions: `deven_`
- 10** Pass-by-Reference Semantics
- 11** Conditional Branching: `if / oth if / oth`
- 12** Iteration: the `during` Construct
- 13** Standard Input: `scan_`
- 14** Standard Output: `print_` and `println_`
- 15** Renaming: the `->` Operator
- 16** Diagnostic State Inspection: `status_`
- 17** Library System
- 18** Portability and Memory Constraints

1 — Language Overview and Architecture

Z is a procedural language with static typing, lexical scoping, call-by-reference argument passing, and a unified loop construct. Programs are organised as a collection of function definitions and an optional header block. Execution begins unconditionally at the entry point `__start()`, which plays the role that *main* plays in C. There is no implicit ordering requirement: function definitions, the header block, and `__start()` may appear in any order within the source file, because the interpreter performs a complete pre-pass over the program before execution begins, resolving all definitions.

The interpreter, Zim, is structured around a single virtual machine value of type VM. This structure holds three parallel tables for scalar variables — one each for integers, floating-point numbers, and characters — and corresponding tables for one-dimensional arrays and two-dimensional matrices of each type. A fixed-size program buffer stores the pre-processed instruction stream; a state stack records the entry points and exit points of all control structures and user-defined functions; a register file of sixteen integer and sixteen character slots supports internal bookkeeping. A scope stack of depth 32 enables the isolation of local variables declared inside function bodies.

The compilation pipeline consists of two phases. In the first phase, *format_code* reads the source text, strips whitespace, normalises line endings, and loads the cleaned instruction stream into the program buffer. In the second phase, *build_state* traverses the buffer and records, for every control structure and function definition, its name, its opening line, and its closing line. This information is stored in the state stack and consulted at runtime to perform forward jumps — for conditionals whose branch is not taken — and backward jumps — for loop iteration. Execution then proceeds line by line through the main parse-and-dispatch loop, *parse()*.

A notable architectural choice is the complete absence of a separate bytecode representation. The instruction stream that Zim executes is the original source text, preprocessed into an array of C strings. Each string is parsed from scratch on every execution. This approach sacrifices throughput for simplicity and, more importantly, for transparency: the debug mode of the interpreter — enabled with the `-dt` flag — prints each instruction as it is dispatched, giving the programmer a complete execution trace at negligible implementation cost.

This architectural simplicity is not a limitation but a deliberate design choice: Z targets environments where firmware size, ROM budget, and the absence of an operating system loader make a compact, auditable interpreter more valuable than raw execution speed.

2 — Lexical Conventions

Every statement in Z is terminated by a colon (:). This includes comments. The colon serves as the universal statement terminator; its omission is a syntax error that will cause the parser to misinterpret the remainder of the source.

Whitespace — spaces, tabs, and blank lines — is ignored everywhere in Z source text, *except* inside double-quoted string literals, where spaces are preserved verbatim. This property means that indentation is purely cosmetic; the programmer may format source code in whatever style aids readability without altering the program's behaviour.

Comments are introduced by the double-solidus sequence `//` and must terminate with the standard statement colon.

```
// This is a valid Z comment:  
// Comments must be terminated with a colon, like every statement:
```

Figure 2.1 — Comment syntax

Identifiers may be composed of letters and digits. The maximum permitted length of an identifier is sixteen characters. Names are case-sensitive. The prefix `__` (two underscores) is reserved for function call invocations and for the entry point `__start()`; it must not be used for variable names.

The token notation `&TYPE&NAME&` is used in specific syntactic positions — primarily in function argument lists and in declaration statements — to unambiguously bind a name to its type. The ampersand delimiters are part of the token syntax and are not operators; they serve purely as delimiters recognised by the parser. In most other contexts, names may be written without token delimiters.

3 — Data Types and Variable Declaration

Z provides three primitive scalar types: integer (**int**), single-precision floating-point (**float**), and character (**char**). All three types may also serve as the element type of one-dimensional arrays and two-dimensional matrices, yielding nine concrete data structure variants in total. There is no implicit coercion between types; the type of a variable is fixed at declaration and does not change.

Scalar variables are declared with the **int_**, **float_**, or **char_** keyword followed by the typed token **&TYPE&NAME&**. All declarations are statements and must be terminated with a colon. Every declared variable is initialised to zero — numerical zero for integers and floats, the null character for chars — at the moment of declaration.

```
int_ &i&particleCount&:  
float_ &l&temperature&:  
char_ &c&statusFlag&:
```

Figure 3.1 — Scalar variable declaration

The type tokens recognised by the parser are as follows: **i** denotes an integer variable; **l** denotes a floating-point variable; **c** denotes a character variable; **n** denotes a numeric literal; **k** denotes a character literal; **s** denotes a string literal or a character array; **a** denotes an array reference (used in function argument declarations); **m** denotes a matrix reference (likewise). The type token is embedded between the first and second ampersand of the **&TYPE&NAME&** notation.

Variables declared inside **__start()** are global in scope: they remain visible to every function called during the execution of **__start()**, and modifications made inside called functions are reflected immediately in the global environment. Variables declared inside a user-defined function introduced with **od_** are local: they are created when the function is entered and destroyed when it returns. Accessing a name that has not been declared in the current scope does not raise a fatal error; instead, it silently evaluates to zero or the null character. This permissive behaviour is intentional and is discussed further in the section on scope.

The maximum number of simultaneously live scalar variables of each type is governed by the compile-time constant **max_number_of_var**, whose default value is 64. This constant, along with the analogous limits for arrays and matrices, may be adjusted in the interpreter source to match the available memory of the target platform. On a microcontroller with 256 KB of SRAM the defaults are conservative and may safely be reduced; on a desktop system they may be expanded freely.

4 — Arrays

A one-dimensional array in Z is declared by placing the size in square brackets immediately after the type token, before the name. The size must be a positive integer literal. Storage for the array is allocated dynamically at the moment of declaration using *calloc*, and all elements are initialised to zero.

```
int_ &i[8]&sampleBuffer&:
float_ &l[4]&coefficients&:
char_ &s[16]&identifier&:
```

Figure 4.1 — Array declaration for the three primitive types

Individual elements are accessed by prefixing the name with the index enclosed in square brackets. The index may be either a non-negative integer literal or the name of an integer variable whose value is used at runtime. Mixed indexing — literal row combined with variable column or vice versa — is fully supported. Access outside the declared bounds produces a diagnostic message and returns a null pointer, which the runtime treats as zero; it does not invoke undefined behaviour as it would in C.

```
int_ &i&offset&:
int_ &i[6]&measurements&:

offset = 2:
[0]measurements = 100:
[offset]measurements = 200:
```

Figure 4.2 — Array element access with literal and variable index

A particularly expressive feature of Z arrays is that virtually any value-producing expression may appear on the right-hand side of an array element assignment: a literal, another variable, another array element, a matrix element, an arithmetic expression involving any of these, or a function call. The following are all valid assignments to an integer array:

```
int_ &i&base&:
int_ &i[4]&results&:

base = 5:
[0]results = 42:
[1]results = base:
[2]results = base + 3:
[3]results = __computeValue(base):
```

Figure 4.3 — Heterogeneous right-hand side in array assignment

When the equality operator `==` is applied to two unindexed array names — that is, to the names without a bracketed index — it compares their *declared sizes*, not their contents. This is a deliberate semantic choice that enables common guard conditions such as confirming that two buffers have compatible dimensions before performing an element-wise

operation. The comparison **arr1 == arr2** evaluates to true if and only if arr1 and arr2 were declared with identical sizes.

5 — Matrices

Two-dimensional matrices are a first-class data structure in Z, declared with two size specifiers placed after the type token. The first specifier gives the number of rows; the second gives the number of columns. Storage is allocated as a single contiguous block of size $\text{rows} \times \text{columns}$, in row-major order, and all elements are initialised to zero. This representation ensures cache-friendly sequential access and admits direct reinterpretation as a flat array when required.

```
int_ &i[4][4]&adjacencyMatrix&:
float_ &l[3][3]&rotationKernel&:
char_ &s[8][16]&tokenGrid&:
```

Figure 5.1 — Matrix declaration

Elements are accessed by supplying both the row and column index, each enclosed in its own pair of square brackets and appearing before the name. As with arrays, either index may be an integer literal or an integer variable, and the two may be mixed freely. This admits the natural programming pattern in which loop variables serve as row and column counters without requiring any intermediate computation.

```
int_ &i&row&:
int_ &i&col&:
int_ &i[3][3]&identityMatrix&:

row = 0:
during(row < 3 ! row++){
  col = 0:
  during(col < 3 ! col++){
    if(row == col){
      [row][col]identityMatrix = 1:
    }
  }
}
```

Figure 5.2 — Matrix initialisation with variable indices

The permissiveness of Z with respect to index expressions extends fully to matrices. The right-hand side of a matrix element assignment may be a literal, a scalar variable, an array element, another matrix element, an arithmetic expression, or the return value of a function call. This uniformity of expression syntax is one of the defining characteristics of Z: the programmer is never required to introduce an auxiliary variable merely to satisfy syntactic restrictions.

Size comparison via unindexed names applies to matrices as well: **mat1 == mat2** evaluates to true if and only if the total element count — $\text{rows} \times \text{cols}$ versus $\text{rows} \times \text{cols}$ — is identical for both operands.

6 — Assignment and Arithmetic Expressions

Assignment in Z is performed with the equals sign. The left-hand side may be a scalar variable, an indexed array element, or an indexed matrix element. The right-hand side may be a literal value, a variable name, an array or matrix element access, an arithmetic binary expression, or a function call. The grammar for the right-hand side is thus deliberately broad: the same syntactic position accepts constants and computed values without restriction.

The four arithmetic operators supported are addition (+), subtraction (-), multiplication (*), and integer division (/). Division by zero is detected at runtime and causes the expression to evaluate to the sentinel value **error_int**, defined as -99, rather than halting the interpreter. Floating-point arithmetic follows the same four operators with single-precision semantics.

```
int_ &i&alpha:
int_ &i&beta:
int_ &i&gamma:

alpha = 12:
beta = 4:
gamma = alpha + beta:
gamma = alpha * beta:
gamma = alpha / beta:
[0]results = alpha - beta:
[1][2]dataGrid = alpha * beta:
```

Figure 6.1 — Arithmetic assignment to scalars, arrays, and matrices

An important property of Z arithmetic is that either operand of a binary expression may be an integer variable, an array element, a matrix element, or a numeric literal. The parser resolves the type of each operand dynamically. There is therefore no distinction at the syntactic level between *variable op variable*, *literal op variable*, and *array element op matrix element*: all are admissible. This eliminates a class of boilerplate assignments that C programmers must write when transferring array data into temporary scalars before performing arithmetic.

7 — Increment and Decrement Operators

Z provides increment and decrement operators that operate on scalar variables and on array elements. The unary forms **varname++** and **varname--** increment or decrement the operand by exactly one. The parameterised forms **varname++N** and **varname--N** add or subtract N, where N may be an integer literal, an integer variable, or the return value of a function call. This last capability — passing a function call as the step amount — is unusual among languages of Z's scope and provides a concise means of driving a variable by a computed quantity without introducing intermediate storage.

```
int_ &i&counter&:
int_ &i&step&:

counter = 10:
step = 3:

counter++:
counter--:
counter++step:
counter--4:
counter++__computeStep():
```

Figure 7.1 — All forms of the increment operator on a scalar variable

The same operators apply to indexed array elements:

```
int_ &i[5]&accumulator&:

[0]accumulator = 7:
[0]accumulator++:
[0]accumulator--2:
[0]accumulator++step:
```

Figure 7.2 — Increment and decrement on array elements

Character arrays exhibit a distinct and specialised behaviour under the **++** and **++N** operators. When applied to an indexed element of a character array, these operators do not perform numeric arithmetic. Instead, **[i]carr++** copies the character at position *i* into position *i+1*, and **[i]carr++N** copies it into position *i+N*. This models the common pattern of propagating a character value along a buffer without an explicit intermediate variable, and is particularly convenient when constructing repetitive character patterns in a string buffer.

```
char_ &s[6]&pattern&:

[0]pattern = 'A':
[0]pattern++:
[0]pattern++3:
```

Figure 7.3 — Character propagation semantics of ++ on a char array element

After the above sequence, position 1 contains 'A' (copied by the first ++) and position 4 contains 'A' (copied by the ++3). Positions 2, 3, and 5 retain their zero-initialised values.

8 — Functions: Declaration and Invocation

User-defined functions are introduced with the keyword **od_** (mnemonic: *open door*), followed by the function name, a parenthesised argument list, and a brace-delimited body. The keyword communicates the designer's intent: a function definition opens a door between the caller's environment and a named, reusable block of behaviour. Functions in Z are *typeless* with respect to their return value: no return-type annotation is required or permitted. A function may return an integer, a floating-point number, or a character; the interpreter infers the type from the value actually returned at runtime.

```
od_ computeNorm(&i&dx& ! &i&dy&){
  int_ &i&sumOfSquares&:
  sumOfSquares = dx * dx:
  sumOfSquares = sumOfSquares + dy * dy:
  deven_ sumOfSquares:
}
```

Figure 8.1 — Function declaration with two integer arguments

Arguments are separated by the exclamation mark (!) within the argument list. Each argument is specified using the typed token syntax. The token type codes for scalar arguments are the same as those used in variable declarations: **i** for integer, **f** for float, **c** for character. Array arguments use the type code **a**; matrix arguments use **m**. A maximum of sixteen arguments per function is enforced by the interpreter, and this limit is adjustable via the compile-time constant **max_parameters**.

Functions are called by prefixing the name with two underscores. Arguments are passed in the same !-delimited form used in the declaration; they are matched positionally. The return value may be ignored, assigned to a scalar variable, stored into an array or matrix element, or used inline as the argument of a print statement or of another function call.

Scalar arguments support *default values*. A default is specified by appending the desired value after the argument token, separated by a space. Default arguments must appear at the end of the argument list; no non-default argument may follow a default argument. Arrays and matrices may not carry defaults. When a function is called with fewer arguments than its declaration specifies, the trailing defaulted parameters receive their declared default values. Omission is all-or-nothing from the point of omission: one may not skip one default and supply the next.

```

od_ translatePoint(&i&x& ! &i&y& ! &i&zOffset& 0){
x = x + zOffset:
y = y + zOffset:
deven_:
}

__start(){
int_ &i&px&:
int_ &i&py&:
px = 3: py = 5:
__translatePoint(px ! py):
__translatePoint(px ! py ! 10):
}

```

Figure 8.2 — Function with a default scalar argument

9 — Return Expressions: `deven_`

The return keyword in Z is **`deven_`**. When followed by a value expression, it terminates the current function and delivers that value to the caller. When written without an argument — **`deven_`**: — it terminates the function with a void result. In either case, `deven_` may appear anywhere inside a function body; upon reaching it, the interpreter immediately exits the function regardless of any surrounding loops or conditionals.

Beyond simple variable or literal returns, Z supports a set of inline unary operations directly in the `deven_` expression. These allow the programmer to apply a transformation to the return value without declaring an additional local variable:

```
deven_ value++:  
deven_ value--:  
deven_ value**:  
deven_ value~~:  
deven_ __otherFunction(arg):
```

Figure 9.1 — Extended `deven_` forms

The `++` and `--` suffixes add or subtract one from the value before returning it. The `**` suffix squares the value in place. The `~~` suffix returns the square root. The function-call form delegates the return to the result of calling another function, enabling single-line functional chains:

```
od_ incrementAndReturn(&i&n&){  
    deven_ n++;  
}  
  
od_ squaredResult(&i&n&){  
    int_ &i&copy&:  
    copy = n;  
    deven_ copy**;  
}  
  
od_ chainedComputation(&i&input&){  
    deven_ __squaredResult(input):  
}
```

Figure 9.2 — Functional chaining via `deven_`

A critical caveat applies to the in-place operators `**` and `~~` when used in conjunction with pass-by-reference arguments. Because all function arguments in Z are aliases to the caller's data, writing **`deven_ n**`**: where `n` is a parameter modifies `n` — and therefore the caller's variable — in place before returning. The correct pattern is to copy the parameter into a local variable and apply the operator to the copy, leaving the caller's value undisturbed.

10 — Pass-by-Reference Semantics

All arguments in Z — scalar variables, arrays, and matrices alike — are passed by reference. A function never receives a copy of its argument; it receives a direct alias to the original storage location in the caller's scope. Every modification made to a parameter inside the called function is immediately visible in the caller upon return.

This semantic propagates transitively across the call chain. If function A passes a variable to function B, and B in turn passes it to function C, then C's modifications are visible in B, and B's modifications — including those propagated from C — are visible in A upon B's return. There is no depth limit on this chain; the alias follows the same storage address through all call frames.

```
od_ shiftRight(&i&value&){
  value++2:
  deven_:
}

od_ shiftAndScale(&i&value&){
  __shiftRight(value):
  value = value * 3:
  deven_ value:
}

__start(){
  int_ &i&signal&:
  signal = 4:
  print_ __shiftAndScale(signal):
}
```

Figure 10.1 — Transitive pass-by-reference across two call frames

After the call, *signal* in `__start()` holds the value 18: `shiftRight` modified it in place to 6 (4 + 2), and `shiftAndScale` then multiplied it by 3. The printed return value and the stored value of *signal* are identical. No copies were made at any point.

Arrays and matrices follow the same rule. Declaring an array argument with the type token **&a&** causes the function to receive a reference to the array in the caller's scope. Writing to any element within the function writes through to the original. This behaviour is not optional and cannot be suppressed; Z does not provide a pass-by-value mode. Programs that require isolation of array data must explicitly copy array contents before passing.

11 — Conditional Branching: if / oth if / oth

Conditional execution in Z is expressed through the **if**, **oth if**, and **oth** constructs, which correspond precisely to the *if*, *else if*, and *else* constructs of C. An **if** block may be followed by any number of **oth if** blocks and optionally terminated with a single **oth** block. Any combination is valid; neither **oth if** nor **oth** is required.

```
if(responseCode == 200){
println_ &s&"OK"&:
}
oth if(responseCode == 404){
println_ &s&"Not Found"&:
}
oth if(responseCode == 500){
println_ &s&"Server Error"&:
}
oth{
println_ &s&"Unknown"&:
}
```

Figure 11.1 — Multi-branch conditional with oth if chains

Conditions are written inside parentheses and support the comparison operators **==**, **<**, **>**, and **^=** (not-equal). An integer variable or array element used alone as a condition — without an explicit operator — is treated as a boolean: non-zero is true, zero is false. This allows compact guard conditions of the form **if(flag)**.

Orphaned **oth if** or **oth** blocks — those not preceded by a matching **if** — are silently skipped. They do not produce an error and do not execute. This permissive behaviour preserves the robustness of the interpreter in incremental development scenarios, where branch structures may temporarily be incomplete.

The interpreter resolves the skip address of each conditional block during the pre-execution `build_state` pass. When a branch is not taken, the instruction pointer jumps directly to the first instruction after the closing brace, without traversing the body. This ensures that even deeply nested or lengthy conditional bodies impose no execution overhead on untaken paths.

12 — Iteration: the during Construct

Z provides a single, unified loop construct: **during**. It subsumes both the counted iteration of a *for* loop and the conditional iteration of a *while* loop. The disambiguation between the two modes is performed automatically by the interpreter based on the form of the argument.

Count-style: When the argument to *during* is a single integer expression — a literal, a variable, or an indexed array element — the loop executes exactly that many times. The count is evaluated once at entry; subsequent modifications to the counted variable have no effect on the remaining iterations.

```
during(8){
println_ &s&"iteration"&:
}

int_ &i&repetitions&:
int_ &i[3]&config&:
repetitions = 5:
[2]config = 3:

during(repetitions){
println_ repetitions:
}

during([2]config){
println_ &s&"config-driven loop"&:
}
```

Figure 12.1 — Count-style during with literal, variable, and array element

Condition-style: When the argument contains a comparison expression — using any of the supported operators — the loop behaves as a *while* loop, re-evaluating the condition at the start of each iteration and exiting as soon as it becomes false. The first element of the argument is the condition; subsequent elements, separated by **!**, are step statements executed at the end of every iteration. Any number of step statements is permitted.

```
int_ &i&index&:
int_ &i&stride&:

index = 0:
stride = 2:

during(index < 10 ! index++stride){
println_ index:
}
```

Figure 12.2 — Condition-style during with a variable step

Step expressions in a condition-style *during* may themselves be increments, decrements, or any other valid Z statement. Multiple steps may be chained, enabling simultaneous

advancement of several loop variables in a single construct without the syntactic overhead that multiple assignment statements would require inside the body.

```
int_ &i&i&:  
int_ &i&j&:  
  
i = 0: j = 20:  
  
during(i < 10 ! i++ ! j--){  
println_ i:  
println_ j:  
}
```

Figure 12.3 — during with two simultaneous step statements

13 — Standard Input: `scan_`

Formatted standard input in Z is provided by the `scan_` statement. It reads from the standard input stream and stores the result into a variable, an array element, or a matrix element, according to the type token supplied. The general syntax is as follows:

```
scan_ # promptText &i&targetVariable& promptText # (stopChar)
scan_ # promptText &s&targetString& promptText # [stopString]
```

Figure 13.1 — `scan_` general form

The `#` delimiters enclose optional prompt text that is printed to standard output before the read occurs. The `&TYPE&NAME&` token identifies the destination. A terminator may be specified in one of two forms: a character enclosed in parentheses — **(stopChar)** — causes reading to halt when that character is encountered in the input stream; a string enclosed in square brackets — **[stopString]** — causes reading to halt when that exact sequence of characters appears. If no terminator is specified, the read proceeds until end of line.

```
int_ &i&age&:
char_ &s[32]&username&:

scan_ # Enter your age: &i&age& Enter your age: #:
scan_ # Enter your name: &s&username& Enter your name: # (newline):
```

Figure 13.2 — `scan_` for an integer and a string with prompt and terminator

The `scan_` statement returns the number of elements successfully read. This return value may be captured in an integer variable and used subsequently to validate the input — for instance, to detect whether a user provided a value or simply pressed Enter without input. This behaviour is directly analogous to the return-value semantics of the C standard library function `scanf`, and the parallel is intentional: Z's `scan_` is designed to be a transparent, safe interface to the same underlying mechanism.

The destination of a `scan_` statement is not restricted to scalar variables. An indexed array element — `&s[i]&buffer&` — is equally valid, enabling the construction of input loops that progressively fill a buffer element by element. The type token in the destination controls the interpretation: **i** for integer, **f** for float, **s** or **c** for character or string.

14 — Standard Output: `print_` and `println_`

Z provides two output primitives. `print_` writes its argument to standard output without appending a newline. `println_` writes its argument followed by a newline character. When called without an argument, `println_` emits a blank line, which is useful for visual formatting of console output.

The argument to either primitive may be any value-producing expression in the Z type system: a scalar variable, a string literal enclosed in the `&s&"..."&` token, an indexed array element, an indexed matrix element, or the return value of a function call. The interpreter infers the required format specifier from the runtime type of the value and dispatches to the appropriate C `printf` format.

```
int_ &i&count&:
float_ &l&ratio&:
char_ &s[12]&label&:
int_ &i[3]&measurements&:

count = 7:
ratio = 3.14:

print_ &s&"Count: "&:
println_ count:
println_ ratio:
print_ &s&"Value: "&:
println_ [0]measurements:
println_ __computeSomething(count):
```

Figure 14.1 — `print_` and `println_` with diverse argument types

The ability to pass a function call directly as the argument to `print_` or `println_` deserves emphasis. This is a natural consequence of Z's expression uniformity: wherever a value is expected, a function call is admissible. The programmer need not assign the function's return value to a variable solely for the purpose of printing it; the call and the print may be composed inline, which both reduces code verbosity and avoids the introduction of auxiliary variables whose sole purpose is to hold a transient result.

15 — Renaming: the -> Operator

The -> operator permanently renames a named entity in the virtual machine. It operates on functions, scalar variables, arrays, and matrices. The syntax places the new name on the left and the current name on the right:

```
newIdentifier -> currentIdentifier:
```

The rename is immediate and permanent. After the statement executes, the entity is accessible only under the new name; the old name is invalidated. Accessing the old name subsequently returns zero or the null character, as though the name had never been declared.

Renaming is applicable to all four categories of named entities. Functions may be renamed to provide a shorter alias or to adapt a library function to a local naming convention without modifying the library source. Variable renames are useful when a single variable plays different conceptual roles at different points in a long computation. Array and matrix renames preserve all element data under the new name.

```
int_ &i&temporaryAccumulator&:  
temporaryAccumulator = 42:  
result -> temporaryAccumulator:  
println_ result:  
  
rett -> retv:  
counter++__rett(5):
```

Figure 15.1 — Variable rename and subsequent use of the new name

The -> operator thus provides a lightweight aliasing facility that is resolved entirely at parse time, with no runtime cost. It is particularly valuable in conjunction with external libraries: a library may expose long, descriptive function names for clarity, and the calling program may introduce short local aliases for ergonomic use in tight loops.

16 — Diagnostic State Inspection: `status_`

The **`status_`** statement prints a complete snapshot of the current virtual machine state to standard output. It enumerates every declared scalar variable of each type, every declared array with its elements, and every declared matrix with its elements, together with their current values. This diagnostic is intended for use during development; it provides in a single statement the information that, in a lower-level language, would require a hand-written set of print statements for every variable in scope.

```
status_:
```

An optional modifier causes `status_` to *clear* the virtual machine state immediately after printing. The accepted modifier keywords are **`cls`**, **`clear`**, and **`clr`**, all equivalent. An additional flag **`y`** may appear anywhere in the statement, in any position relative to the clear keyword, and its presence confirms the intent to clear:

```
status_ cls:
status_ clear:
status_ clr:
status_ cls y:
status_ y clear:
```

Figure 16.1 — All accepted forms of the clearing `status_` statement

After a clearing `status_`, all variables, arrays, and matrices that were live at the time of the call are destroyed. Their storage is reclaimed and their names are no longer resolvable. Only entities declared after the `status_` call will exist in the subsequent execution. This facility is useful in programs that operate in distinct phases: a setup phase may compute and print intermediate results and then clear all working storage before the main computation phase begins, ensuring that no residual values interfere with the principal algorithm.

17 — Library System

Z supports external library modules, stored as **.Zlib** files. A library file is structurally identical to a **.Zim** source file: it contains function definitions written in ordinary Z syntax. The distinction is purely organisational — a **.Zlib** file is intended to define reusable functions that are independent of any particular program.

Libraries are attached to a program at invocation time on the command line, using the **-libname.Zlib** convention. The interpreter, upon receiving one or more library arguments, merges each library into the source file before the `format_code` and `build_state` passes execute. The merge is performed by prepending the source content to the library content in a temporary file, so that the resulting program buffer contains all library definitions alongside the main source, fully resolved, before execution begins.

```
gcc Zinterpreter.c -o Zinterpreter -lm
./Zinterpreter -df mainProgram.Zim -lstdmath.Zlib
./Zinterpreter -df mainProgram.Zim -lstdmath.Zlib -lioutils.Zlib
```

Figure 17.1 — Invoking the interpreter with one or more library files

After execution, the interpreter restores the original source file by reverting the merge: the temporary combined file is discarded and the original **.Zim** file is reinstated. The library is thus consumed non-destructively; the source file is unchanged after execution, and the same library may be used with any number of source files without modification.

The order in which multiple libraries are specified on the command line is the order in which their definitions are prepended. Inter-library dependencies must be arranged accordingly; a library that calls functions defined in another library must be listed after that library on the command line.

18 — Portability and Memory Constraints

The portability of Z is a first-order design objective, not an afterthought. The interpreter is a single C source file with no dependency beyond the standard C library and the mathematics library **libm**. It compiles without warnings under gcc and clang on Linux, macOS, and Windows, and is explicitly designed to compile on the ESP32 Arduino toolchain, which targets an Xtensa LX6 core running FreeRTOS with a SRAM budget of approximately 320 KB.

The entire virtual machine is declared as a single global variable of type **VM** and initialised with **memset** to zero at startup. No heap allocation occurs during the startup sequence; the fixed-size portions of the VM — the variable tables, the program buffer, the state stack, and the register file — are all statically sized within the struct. Dynamic allocation occurs only when arrays or matrices are declared at runtime, through **calloc** calls bounded by the declared sizes.

The memory footprint of the interpreter at startup, measured on Linux x86-64 with default compile-time constants, is approximately 284 KB. This figure is dominated by the program buffer (512 lines × 1024 bytes per line) and the state stack. On memory-constrained targets, either limit may be reduced: halving the program buffer to 256 lines reduces the base footprint by approximately 128 KB, which is significant on a microcontroller.

All compile-time capacity constants — **max_number_of_var**, **max_number_of_array**, **max_number_of_cell**, **max_name_letter**, **max_parameters**, and **max_number_of_reg** — are gathered at the top of the source file and are the only values that need modification when porting to a new target. No other change to the interpreter source is required. A port to a new platform consists literally of adjusting these constants to match the available memory and recompiling with the target toolchain.

This architecture positions Z uniquely among interpreted languages: it is simultaneously a general-purpose scripting language suitable for desktop use and a firmware-embeddable scripting engine suitable for microcontroller deployment, with the same source code, the same syntax, and the same semantics on both. A program developed and tested on a workstation will run identically on an ESP32, subject only to the memory limits imposed by the target's capacity constants. No cross-compilation step, no virtual machine image format, and no separate tool chain are required; the interpreter itself is the deployment artefact.

Z is a language that scales with its host. On a workstation, it operates with generous buffer sizes and provides full debug-trace capability. On a microcontroller, the same interpreter, recompiled with reduced constants, occupies a fraction of available SRAM and runs silently. The programmer's code travels with it, unchanged, between both worlds. That is the essential promise of Z.

— *End of Reference Manual* —